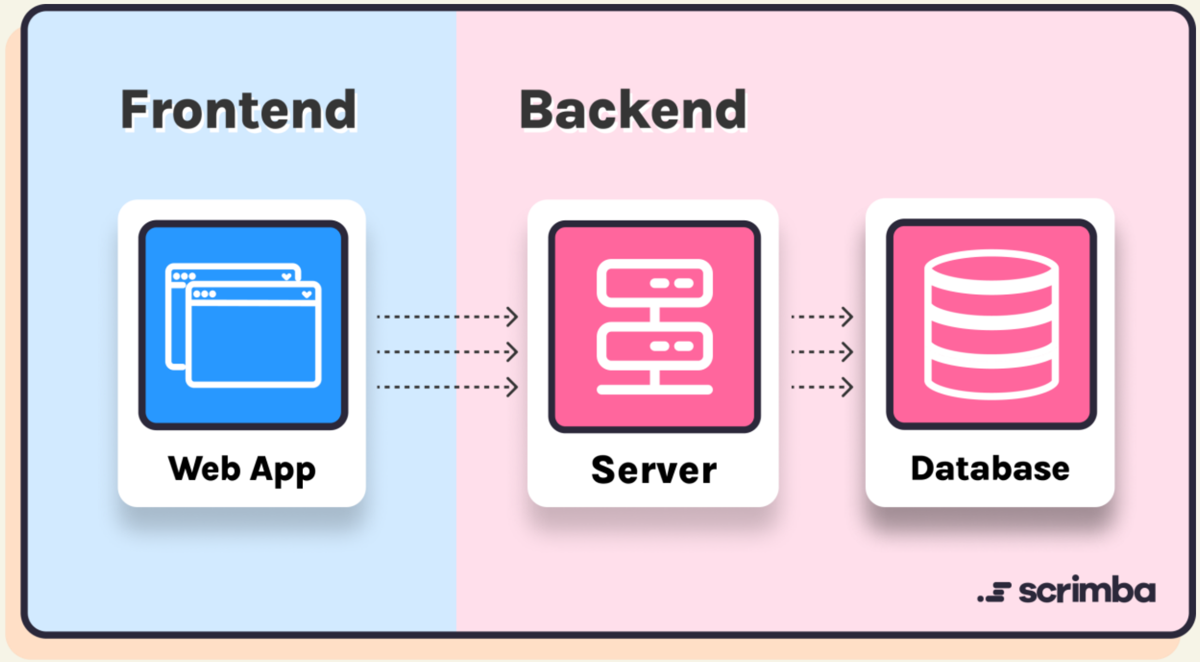
# Modern Backend Development

Chapter 1 — Introduction to Backend Development

Firebase • MongoDB • PHP • MySQL

MODERN
BACKEND
DEVELOPMENT
BUILD • CONNECT • DEPLOY
KEY CONCEPTS
APIs & Endpoints
Databases (SQL & NoSQL)
Authentication & Security
Data Processing
Cloud & Deployment
CLIENT
Web / Mobile
API
{ API }
SERVER
Backend Logic
TECHN
EXPLORER
server.js
database.js
backend-project
config
database.js
routes
userRoutes.js
controllers
userController.js
models
User.js
middleware
auth.js
server.js
.env
package.json
conct express = require('express');
const app = express();
const db = require('./config/database');
// Middleware
app.use(express.json());
// Routes
app.use('/api/users', require('./routes/userRoutes'));
// Server
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
console.log('Server running on port ${PORT}');
});
Ln.1, Col 1
Spaces: 2
UTF-8
LF
JavaScript
</CODE>
Frontend
Auth
API
Database
Server


Frontend
Backend
Web App
Server
Database
scrimba

# What is Backend?

- • Hidden logic behind applications
- • Handles data and authentication
- • Connects frontend with database
- • Processes requests and responses

- Real Example:
- Instagram stores accounts, posts, likes, and messages using backend systems.

# Frontend vs Backend

- Frontend:
- • UI Design
- • HTML, CSS, JavaScript
- • What users see

- Backend:
- • Database
- • APIs
- • Authentication
- • Business Logic

# How Websites Work

Flow:

User → Frontend → Backend Server → Database

- User sends request
- Backend processes data
- Database stores/fetches information
- Response returns to user

# Login System Flow

- Step 1: User enters email & password
- Step 2: Backend receives request
- Step 3: Database verification
- Step 4: Success or error response

- Important:
- Frontend never directly accesses database.

# What is API?

- API = Communication bridge
- • Connects frontend and backend
- • Transfers data using JSON
- • Used in websites and mobile apps
- Examples:
- Instagram API
- Weather API
- Payment API

# API Methods

- GET → Fetch Data
- POST → Send Data
- PUT → Update Data
- DELETE → Remove Data

- These methods are used in REST APIs.

# JSON Basics

- Example JSON:
- {
- "name": "Arshad",
- "course": "Backend"
- }

- • Key-value structure
- • Lightweight data format
- • Used in APIs and backend communication

# SQL vs NoSQL

- SQL:
- • Tables and rows
- • Structured data
- • Example: MySQL

- NoSQL:
- • JSON documents
- • Flexible structure
- • Example: MongoDB, Firebase

# MySQL Database Structure

- • Tables
- • Rows
- • Columns
- • Primary Keys

- Best for:
- Banking systems
- Management systems
- Structured applications

# MongoDB Document Structure

- • Collections
- • Documents
- • JSON storage

- Best for:
- Chat applications
- Social media apps
- Flexible backend systems

# Backend Security

- Authentication

- Authorization

- Password protection

- Backend security layer

Backend protects database from unauthorized access.

# Real-World Backend Examples

Instagram → User accounts & feeds

WhatsApp → Messaging system

Amazon → Orders & payments

YouTube → Videos & recommendations

# Practical Session

- Practical 1:
- Create simple JSON response.
- Practical 2:
- Understand request and response cycle

- Practical 3:
- Observe APIs using browser inspect tools

# Conclusion

Students learned:

- Backend fundamentals
- APIs
- JSON
- SQL vs NoSQL
- Client-server architecture

Next Chapter:

Firebase Backend Fundamentals